

TUGAS BESAR IF2224

MILESTONE 2

SYNTAX ANALYSIS



Kelompok std_abs

Ega Luthfi Rais	13524115
Salman Faiz Assidqi	13524134
Ahmad Fauzan Putra	13524141
Leonardus Brain Fatolosja	13524146

Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI	2
BAB 1 TEORI SINGKAT	3
1.1 Parser dan Parse Tree	3
1.2 Recursive Descent (Top-Down)	4
1.3 Syntax Analysis	5
BAB 2 PERANCANGAN & IMPLEMENTASI	7
2.1 Rancangan Program Recursive Descent	7
2.2 Implementasi Recursive Descent	7
2.2.2 Struktur Program	7
2.2.3 Mekanisme Kerja Parser (Recursive Descent)	7
2.2.4 Pembentukan Tree	8
2.2.5 Output Tree	8
2.2.6 Catatan Implementasi	8
2.2.7 Implementasi Program dalam C++	8
BAB 3 PENGUJIAN	9
3.1 Pengujian 1 (input-1.txt):	9
3.2 Pengujian 2 (input-2.txt):	11
3.3 Pengujian 3 (input-3.txt):	16
3.4 Pengujian 4:	20
3.5 Pengujian 5:	25
BAB 4 KESIMPULAN DAN SARAN	33
4.1 Kesimpulan	33
4.2 Saran	33
LAMPIRAN	34
REFERENSI	35

BAB 1 TEORI SINGKAT

1.1 Parser dan Parse Tree

Parser merupakan salah satu bagian penting dalam proses compiler karena bagian memeriksa apakah susunan token dari lexer sudah membentuk program yang benar. Pada tahap sebelumnya, lexical analyzer memecah source code menjadi token, misalnya keyword, identifier, operator, separator, dan konstanta. Akan tetapi, token-token tersebut belum cukup untuk menyatakan bahwa sebuah program sudah benar. Parser kemudian membaca urutan token tersebut dan mencocokkannya dengan grammar bahasa yang digunakan.

Dalam tugas besar ini, parser digunakan untuk bahasa Arion. Masukan parser adalah list token hasil lexical analysis, sedangkan keluarannya adalah parse tree. Jadi, parser tidak lagi berurusan dengan karakter mentah seperti huruf per huruf, tetapi sudah bekerja pada bentuk token. Misalnya, potongan program yang berisi nama program, deklarasi variabel, dan statement akan diperiksa berdasarkan aturan grammar seperti <program>, <program-header>, <declaration-part>, dan <compound-statement>.

Parse tree adalah bentuk pohon yang memperlihatkan bagaimana suatu program dibangun dari aturan grammar. Node paling atas biasanya merepresentasikan struktur paling umum, misalnya <program>. Dari node tersebut, pohon bercabang ke bagian-bagian yang lebih kecil seperti header program, bagian deklarasi, blok statement, expression, term, factor, dan token terminal. Dengan bentuk pohon ini, struktur program menjadi lebih mudah dilihat karena hubungan antarbagian program dapat ditampilkan secara bertingkat.

Pada milestone ini, parse tree menjadi keluaran utama yang harus dihasilkan oleh parser. Parse tree berbeda dari abstract syntax tree atau AST. Parse tree biasanya masih menampilkan struktur grammar secara lengkap, termasuk simbol terminal dan non-terminal. Sementara itu, AST lebih ringkas karena hanya menyimpan informasi yang dianggap penting untuk tahap berikutnya. Berdasarkan spesifikasi milestone ini, parser perlu menghasilkan struktur yang cukup lengkap dan sesuai dengan grammar Arion.

Selain membangun parse tree, parser juga harus mampu mendeteksi kesalahan sintaks. Contohnya adalah ketika token semicolon hilang, keyword begin tidak memiliki pasangan end, tanda kurung tidak lengkap, atau statement tidak sesuai pola yang ditentukan grammar. Dengan adanya parser, kesalahan seperti ini dapat diketahui lebih awal sebelum program masuk ke tahap semantic analysis.

1.2 Recursive Descent (Top-Down)

Metode yang digunakan dalam milestone ini adalah Recursive Descent. Recursive Descent termasuk metode top-down parsing karena prosesnya dimulai dari simbol awal grammar, lalu diturunkan sedikit demi sedikit sampai cocok dengan token input. Dalam konteks bahasa Arion, proses parsing dimulai dari node `<program>`, kemudian parser memeriksa bagian turunannya seperti `<program-header>`, `<declaration-part>`, `<compound-statement>`, dan period.

Ide utama Recursive Descent adalah membuat satu fungsi untuk setiap non-terminal dalam grammar. Misalnya, aturan `<program>` dapat dibuat sebagai fungsi `parseProgram()`, aturan `<program-header>` dapat dibuat sebagai `parseProgramHeader()`, dan aturan `<statement>` dapat dibuat sebagai `parseStatement()`. Setiap fungsi bertugas memeriksa token yang sedang dibaca, memastikan token tersebut sesuai aturan, lalu memanggil fungsi lain apabila grammar membutuhkan struktur yang lebih kecil.

Pendekatan ini cukup nyaman digunakan untuk tugas parser karena alur kodenya mirip dengan grammar. Ketika grammar menyatakan bahwa program terdiri atas header, deklarasi, compound statement, dan period, maka fungsi `parseProgram()` juga dapat ditulis dengan urutan pemeriksaan yang sama. Hal ini membuat kode parser lebih mudah dibaca, dibagi ke dalam modul, dan diperiksa ketika terjadi error.

Dalam implementasinya, parser biasanya memiliki penunjuk token saat ini. Ketika token yang dibaca sesuai dengan token yang diharapkan, parser melakukan proses consume atau advance ke token berikutnya. Namun jika token tidak sesuai, parser akan mengeluarkan pesan error. Pesan error sebaiknya tidak hanya menyatakan bahwa program salah, tetapi juga menjelaskan token apa yang ditemukan dan token apa yang seharusnya muncul. Dengan begitu, pengguna lebih mudah memperbaiki source code atau input token yang salah.

- a. Lexical Analysis. Tahap ini mengubah source code menjadi token-token. Lexer mengenali satuan seperti keyword, identifier, operator, separator, integer constant, real constant, character constant, dan string. Hasil lexical analysis inilah yang menjadi input untuk parser.
- b. Syntax Analysis / Parsing. Tahap ini memeriksa susunan token berdasarkan grammar. Parser memastikan bahwa token-token tidak hanya valid secara individu, tetapi juga tersusun sebagai program yang benar. Jika valid, parser membangun parse tree. Jika tidak valid, parser memberikan syntax error.
- c. Semantic Analysis. Tahap ini dilakukan setelah struktur sintaks dinyatakan benar. Semantic analysis memeriksa makna program secara lebih dalam, misalnya

apakah variabel sudah dideklarasikan, apakah tipe data sesuai, dan apakah pemanggilan fungsi atau prosedur sudah benar.

- d. Intermediate Code Generation. Tahap ini membuat representasi antara dari program. Representasi tersebut belum tentu berupa kode mesin, tetapi lebih mudah diproses untuk optimasi atau penerjemahan ke tahap berikutnya.
- e. Execution. Tahap ini adalah tahap ketika program akhirnya dijalankan, baik melalui hasil kompilasi, virtual machine, maupun proses lain yang sesuai dengan rancangan compiler.

1.3 Syntax Analysis

Syntax analysis merupakan tahap yang memeriksa struktur sintaks dari token-token hasil lexical analysis. Jika lexical analysis bertugas mengenali token, maka syntax analysis bertugas memastikan bahwa token tersebut tersusun sesuai aturan grammar. Sebagai contoh, token `programs`, `ident`, dan `semicolon` dapat membentuk `<program-header>` apabila muncul dalam urutan yang benar. Jika urutannya salah atau ada token yang hilang, parser akan menolak input tersebut sebagai kesalahan sintaks.

Pada bahasa Arion, syntax analysis dilakukan dengan membaca list token dari hasil milestone sebelumnya. Token-token tersebut kemudian dibandingkan dengan aturan produksi grammar yang sudah ditentukan dalam spesifikasi. Parser harus mampu mengenali struktur besar seperti `<program>`, `<declaration-part>`, dan `<compound-statement>`, serta struktur yang lebih kecil seperti `<assignment-statement>`, `<if-statement>`, `<expression>`, `<variable>`, dan `<factor>`.

Peran syntax analysis sangat penting karena tahap ini menjadi jembatan antara lexer dan semantic analyzer. Lexer hanya menghasilkan token, sedangkan semantic analyzer membutuhkan struktur program yang lebih jelas. Parser mengisi bagian tengah tersebut dengan menyusun token menjadi parse tree. Dengan parse tree, compiler dapat mengetahui bagian mana yang merupakan deklarasi, bagian mana yang merupakan statement, dan bagian mana yang merupakan ekspresi.

Dalam tugas ini, Recursive Descent digunakan untuk menjalankan syntax analysis. Setiap aturan grammar diimplementasikan menjadi fungsi parser yang saling memanggil. Misalnya, ketika parser membaca compound statement, parser akan memeriksa token `beginsy`, kemudian membaca statement-list, lalu memastikan ada token `endsy`. Pola seperti ini membuat proses parsing lebih sistematis karena setiap bagian grammar memiliki fungsi yang bertanggung jawab atas validasinya sendiri.

Syntax analysis juga membantu menghasilkan output yang lebih terstruktur. Jika program valid, parser akan menghasilkan parse tree yang memperlihatkan hubungan

antara node satu dengan node lainnya. Misalnya, root <program> dapat memiliki anak berupa <program-header>, <declaration-part>, <compound-statement>, dan period. Di dalam compound statement, parser dapat membentuk node statement-list yang berisi assignment statement, conditional statement, loop statement, atau procedure/function call sesuai input.

Dengan demikian, syntax analysis tidak hanya berfungsi untuk menentukan apakah input diterima atau ditolak. Tahap ini juga membangun representasi program dalam bentuk parse tree yang dapat dipakai pada tahap selanjutnya. Parser yang baik harus mampu membaca token dengan benar, mengikuti grammar secara konsisten, menghasilkan parse tree yang rapi, serta memberikan pesan error yang jelas ketika ditemukan kesalahan sintaks.

BAB 2 PERANCANGAN & IMPLEMENTASI

2.1 Rancangan Program Recursive Descent

Program parser bahasa Arion dirancang menggunakan pendekatan Recursive Descent yang bersifat top-down. Grammar bahasa Arion di-hardcode ke dalam kode sumber sehingga setiap non-terminal pada grammar memiliki satu fungsi parser tersendiri. Fungsi-fungsi tersebut saling memanggil sesuai aturan produksi, dimulai dari simbol awal <program> dan diturunkan hingga simbol terminal berupa token.

Alur kerja program secara garis besar adalah: (1) pengguna memberikan file sumber Arion atau file token hasil lexical analysis, (2) lexer menghasilkan vector<Token>, (3) parser membaca vector tersebut secara sekuensial melalui pointer posisi dan membangun ParseNode root berlabel <Program>, (4) parse tree dicetak ke terminal dan ke file output txt. Input parser dapat berupa source code langsung maupun file berisi token siap pakai hasil milestone 1 melalui modul tokenReadWrite.

Berikut adalah keseluruhan aturan produksi yang di-hardcode ke dalam parser. Notasi yang digunakan: "|" untuk pilihan, "()" untuk opsional, "()*" untuk nol-atau-lebih, "()*+" untuk satu-atau-lebih, dan "+" untuk konkatenasi urutan.

Non-terminal	Aturan Produksi	Penjelasan Singkat
<program>	program-header + declaration-part + compound-statement + period	Simbol awal grammar; sebuah program lengkap terdiri atas header, bagian deklarasi, blok statement utama, dan diakhiri tanda titik.
<program-header>	programsy + ident + semicolon	Baris pembuka program: keyword program, nama program, lalu titik koma.
<declaration-part>	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Sekumpulan deklarasi opsional yang harus berurutan: const, type, var, lalu subprogram (procedure/function). Tiap kelompok boleh tidak ada.
<const-declaration>	constsy + (ident + eql + constant + semicolon)+	Mendeklarasikan satu atau lebih konstanta dengan format nama = nilai; setelah keyword const.
<constant>	charcon string [(plus minus)? + (ident intcon realcon)]	Nilai literal sebuah konstanta: karakter, string, bilangan bulat, bilangan riil, atau identifier konstanta lain, opsional dengan tanda + / -.
<type-declaration>	typesy + (ident + eql + type + semicolon)+	Mendefinisikan satu atau lebih tipe baru beserta aliasnya setelah keyword type.
<var-declaration>	varsy + (identifier-list + colon + type + semicolon)+	Mendeklarasikan satu atau lebih variabel dengan tipe tertentu setelah keyword var.
<identifier-list>	ident + (comma + ident)*	Daftar satu atau lebih identifier yang dipisahkan koma.

<type>	ident array-type range enumerated record-type	Sebuah tipe dapat berupa nama tipe (mis. integer), array, range/subrange, enumerasi, atau record.
<array-type>	arraysy + lbrack + (range ident) + rbrack + ofsy + type	Tipe array dengan indeks berupa range/nama tipe diikuti tipe elemen.
<range>	constant + period + period + constant	Subrange: dua konstanta yang dipisahkan oleh tanda .. (dua periode).
<enumerated>	lparent + ident + (comma + ident)* + rparent	Tipe enumerasi: daftar identifier dalam kurung, dipisah dengan koma.
<record-type>	recordsy + field-list + endsy	Tipe record yang dibuka keyword record, berisi daftar field, dan ditutup end.
<field-list>	field-part + (semicolon + field-part)*	Satu atau lebih field part yang dipisahkan titik koma di dalam record.
<field-part>	identifier-list + colon + type	Sebuah field record: daftar identifier dengan tipe yang sama.
<subprogram-declaration>	procedure-declaration function-declaration	Deklarasi subprogram dapat berupa procedure atau function.
<procedure-declaration>	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon	Procedure: nama, parameter opsional, lalu blok body diapit titik koma.
<function-declaration>	functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon	Function: seperti procedure tetapi dengan tipe kembalian setelah tanda titik dua.
<block>	declaration-part + compound-statement	Body subprogram: deklarasi lokal lalu compound statement.
<formal-parameter-list>	lparent + parameter-group + (semicolon + parameter-group)* + rparent	Daftar parameter formal di dalam kurung, antar group dipisah titik koma.
<parameter-group>	identifier-list + colon + (ident array-type)	Satu group parameter: daftar identifier dengan satu tipe (nama tipe atau array).
<compound-statement>	beginsy + statement-list + endsy	Blok statement yang diapit begin dan end.
<statement-list>	statement + (semicolon + statement)*	Urutan statement yang dipisahkan titik koma.
<statement>	(assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call)?	Sebuah statement, atau statement kosong (sehingga ;; tetap valid).
<variable>	ident + (component-variable)*	Sebuah variabel: nama, opsional diikuti akses elemen array atau field record berkali-kali.
<component-variable>	(lbrack + index-list + rbrack) (period + ident)	Akses komponen variabel: indeks array [...] atau field record .namaField.
<index-list>	(intcon charcon ident) + (comma + index-list)*	Daftar indeks array yang dipisahkan koma; tiap indeks berupa intcon, charcon, atau identifier.
<assignment-statement>	variable + becomes + expression	Statement penugasan nilai ekspresi ke variabel dengan operator :=.
<if-statement>	ifsy + expression + thensy + statement + (elsesy + statement)?	Pencabangan kondisional dengan cabang else opsional.
<case-statement>	casesy + expression + ofsy + case-block + endsy	Pencabangan multi-cabang berdasarkan nilai ekspresi, ditutup end.

<case-block>	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*	Satu cabang case: satu/lebih konstanta label diikuti : dan statement, dapat dilanjutkan cabang lain.
<while-statement>	whilesy + expression + dosy + statement	Pengulangan pre-test: jalankan statement selama ekspresi bernilai true.
<repeat-statement>	repeatsy + statement-list + untilsy + expression	Pengulangan post-test: jalankan statement-list sampai ekspresi bernilai true.
<for-statement>	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + statement	Pengulangan dengan counter dari nilai awal sampai akhir, naik (to) atau turun (downto).
<procedure/function-call>	ident + (lparent + parameter-list? + rparent)?	Pemanggilan procedure atau function, dengan atau tanpa argumen.
<parameter-list>	expression + (comma + expression)*	Daftar argumen aktual yang dipisahkan koma pada saat pemanggilan.
<expression>	simple-expression + (relational-operator + simple-expression)?	Ekspresi dengan satu operator relasional opsional di tingkat paling luar.
<simple-expression>	(plus minus)? + term + (additive-operator + term)*	Tingkat operator aditif (+, -, or) dengan tanda unary opsional di depan.
<term>	factor + (multiplicative-operator + factor)*	Tingkat operator multiplikatif (*, /, div, mod, and).
<factor>	ident intcon realcon charcon string (lparent + expression + rparent) (notsy + factor) procedure/function-call variable	Atom ekspresi: literal, identifier/variable, ekspresi dalam kurung, negasi not, atau pemanggilan function.
<relational-operator>	eql neq gtr geq lss leq	Operator pembandingan: =, <, >, >=, <=, <=.
<additive-operator>	plus minus orsy	Operator level penjumlahan/pengurangan dan logika or.
<multiplicative-operator>	times rdiv idiv imod andsy	Operator level perkalian/pembagian dan logika and (*, /, div, mod, and).

2.2 Implementasi Recursive Descent

2.2.1 Gambaran Umum

Parser diimplementasikan sebagai kelas Parser dalam berkas parser.hpp. Kelas ini menyimpan daftar token yang diterima dari lexer pada atribut tokens dan pointer posisi pos. Seluruh fungsi parser merupakan member function privat kecuali konstruktor dan method publik parse(). Method parse() menjadi entry point yang akan memulai proses parsing pada level <program>. Untuk memudahkan debugging, terdapat modul pendukung di folder debugger/ yang berisi ASTVisualizer (menghasilkan file .dot untuk digambar dengan Graphviz), parserTrace (makro log jejak eksekusi fungsi parser), dan tokenDumper (mencetak seluruh token). Debugger tersebut dapat diaktifkan melalui makro tanpa mengubah alur utama program.

2.2.2 Struktur Program

a. utils/

Berisi implementasi fungsi untuk mencetak tree yang sudah dibuat (printTree.hpp), membaca file-file yang berisi token hasil dari lexical analysis (tokenReadWrite.hpp), menulis hasil ke dalam file (writeFile.hpp), dan case-case command yang akan dipanggil dalam fungsi main (command.hpp)

b. syntax-analysis/

Berisi definisi dari fungsi-fungsi untuk melakukan parsing dengan menggunakan recursive descent (parser.hpp) dan definisi untuk struktur data tree yang berada di dalam program ini (parseTreeNode.hpp). Implementasi parser.hpp dibagi ke dalam delapan (8) file untuk mencegah menumpuknya implementasi kode di dalam satu file. File-file implementasi terbagi ke dalam beberapa jenis, yaitu antara lain:

- parser.cpp : fungsi parse() yang akan menjadi fungsi utama dan berisi fungsi-fungsi dasar untuk menunjang parsing (peek(), peekahead(), expect(), match(), advance(), dan error()).
- parseProgram.cpp : implementasi parseProgramHeader() yang membaca programsy, ident, dan semicolon, serta parseBlock() untuk body subprogram (deklarasi lokal lalu compound statement).
- ParserDeclaration.cpp : parseDeclarationPart() dan turunannya untuk mem-parse const, type, var, dan subprogram declaration; termasuk parseProcedureDeclaration(), parseFunctionDeclaration(), parseFormalParameterList(), dan parseParameterGroup().
- ParserType.cpp : parseType() untuk berbagai jenis tipe, parseArrayType(), parseRange(), parseEnumerated(), parseRecordType(), parseFieldList(), dan parseFieldPart().
- ParserStatement.cpp : parseCompoundStatement(), parseStatementList(), parseStatement(), serta seluruh jenis statement (assignment, if, case, while, repeat, for) beserta parseCaseBlock().
- ParserVariable.cpp : parseVariable() dan parseComponentVariable() untuk akses elemen array (lbrack/rbrack) dan field record (period).
- ParserExpression.cpp : parseExpression(), parseSimpleExpression(), parseTerm(), parseFactor(), serta parseProcedureFunctionCall() dan parseParameterList().
- ParserHelper.cpp : fungsi-fungsi pembantu yang dipakai di banyak tempat: parseIdentifierList(), parseConstant(), parseIndexList(), parseRelationalOperator(), parseAdditiveOperator(), dan parseMultiplicativeOperator().

c. Main.cpp

Berfungsi sebagai *driver* program yang menerima *file* input, menjalankan Lexical Analysis atau/dan Syntax Analysis, dan menampilkan hasil token atau/dan tree.

d. debugger/

Berisi modul bantu pengembangan parser: tokenDumper.hpp untuk mencetak seluruh token, parserTrace.hpp dengan makro TRACE() yang melacak pemanggilan fungsi parser, dan astVisualizer.hpp yang mengekspor parse tree ke format DOT.

2.2.3 Mekanisme Kerja Parser (Recursive Descent)

a. Inisiasi

Penerimaan token hasil dari lexical analysis dalam bentuk vector/list yang nantinya akan diproses dalam parser. Fase inisiasi memastikan data yang dibutuhkan oleh parser sudah sesuai dan valid sehingga parser dapat berfungsi dengan baik.

b. Fungsi utama - (parser())

Fungsi utama akan memanggil masing-masing bagian, yaitu program header, declaration part, dan compound statement. Fungsi ini yang merupakan jalur masuk untuk melakukan parsing.

c. Fungsi pendukung

Parser menyediakan sekumpulan fungsi primitif yang menjadi dasar seluruh parsing: *peek()* mengembalikan token saat ini tanpa menggesernya, *peekAhead(n)* melihat token ke-n di depan untuk membantu pengambilan keputusan ketika first-set dua aturan produksi sama, *advance()* menggeser pos ke token berikutnya, *match(type)* memeriksa apakah token saat ini bertipe tertentu, dan *expect(type)* menghasilkan ParseNode terminal jika cocok atau memanggil *error()* jika tidak.

d. Alur pengecekan token

Untuk setiap aturan produksi, fungsi parser mengikuti pola "peek - putuskan cabang - expect/recursive-call - advance". Sebagai contoh, *parseVarDeclaration()* terlebih dahulu meng-expect VARSY, lalu selama token berikutnya adalah IDENT, ia akan mem-parse identifier-list, COLON, type, dan SEMICOLON secara berulang. Pendekatan ini membuat struktur fungsi parser hampir 1:1 dengan grammar.

e. Penanganan error

Ketika token yang diharapkan tidak ditemukan, *expect()* memanggil *error()* yang mencetak pesan sintaks error informatif lalu mengembalikan ParseNode berlabel "error". Node error tetap ditempatkan dalam parse tree sehingga posisi kesalahan dapat dilihat secara visual tanpa membuat program crash, dan parser dapat melanjutkan parsing untuk mendeteksi error berikutnya.

2.2.4 Pembentukan Tree

Setiap node dibuat melalui fungsi *expect()* yang mengembalikan nilai *parseNode()*. Fungsi ini menerima ekspektasi token, membandingkannya dengan token yang sedang dicek, dan mengembalikan node jika cocok atau mengembalikan node error jika tidak cocok. Fungsi utama adalah *parse()* yang memuat keseluruhan program parser.

2.2.5 Output Tree

Hasil yang dihasilkan adalah tree yang akan ditampilkan di terminal atau/dan di file output. Tree yang dicetak didapat dari struktur data tree yang telah dibuat sebelumnya.

2.2.6 Catatan Implementasi

Ada beberapa hal penting dalam implementasi, yaitu:

- a. Pengimplementasian rule <while-statement> dan <for-statement>. Parser mengikuti spesifikasi grammar, yaitu body loop berupa <statement> tunggal. Sesuai QnA akan di perbaiki pada milestone selanjutnya.

2.2.7 Implementasi Program dalam C++

Berikut merupakan tampilan program secara garis besar.

```

src > syntax-analysis > parser.hpp > Parser
1  #ifndef PARSE_HPP
4  #include <vector>
8
9  class Parser {
10 public:
11     Parser(const std::vector<Token>& tokenList);
12     ParseNode parse();
13
14     // Debugging
15     ParseNode testDummyTree();
16 private:
17     // Atribut
18     std::vector<Token> tokens;
19     int pos;
20
21     // Private Method
22     Token peek() const;
23     Token peekAhead(int n) const;
24     void advance();
25     bool match(TokenType type) const;
26     ParseNode expect(TokenType type);
27     ParseNode error(const std::string& msg) const;
28
29     std::string tokenToLabel(const Token& t) const;
30
31     // Deklarasi Fungsi Parser (Spesifik)
32     // 1. Header (Program) [parserProgram.cpp]
33     ParseNode parseProgramHeader();
34     ParseNode parseBlock();
35
36     // 2. Declaration [parserDeclaration.cpp]
37     ParseNode parseDeclarationPart();
38     ParseNode parseConstDeclaration();
39     ParseNode parseTypeDeclaration();
40     ParseNode parseVarDeclaration();
41     ParseNode parseSubprogramDeclaration();
42     ParseNode parseProcedureDeclaration();

```

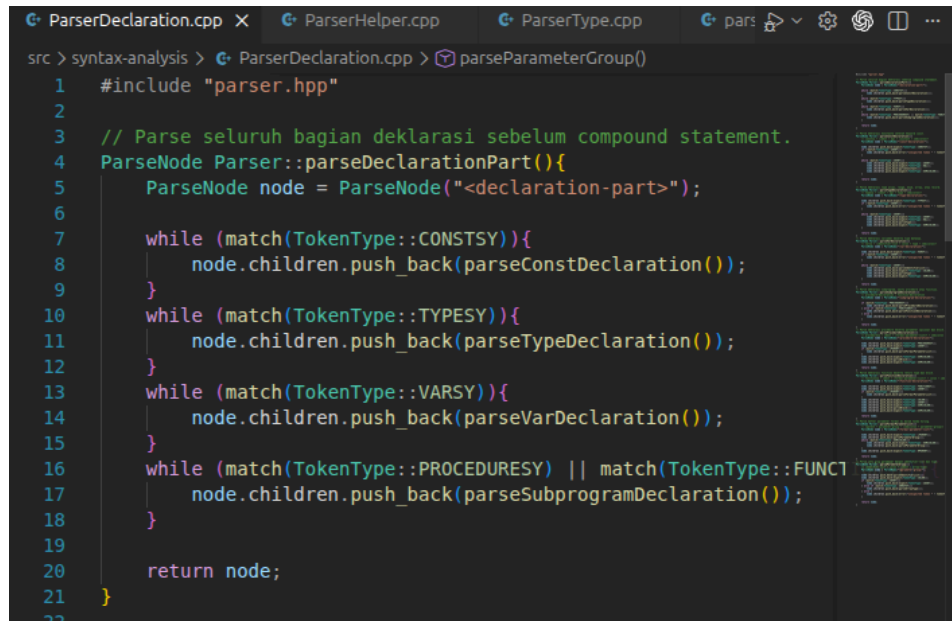
Gambar 1. Deklarasi Parser.hpp

```

src > syntax-analysis > parser.cpp > advance()
46 ParseNode Parser::expect(TokenType type) {
47     if (match(type)) {
48         return node;
49     } else {
50         string expectedStr = tokenToString(type);
51         string foundStr = tokenToString(peek().type);
52         return error("unexpected token " + foundStr + ", expected " + expectedStr);
53     }
54 }
55
56 // Cetak syntax error dan kembalikan node error.
57 ParseNode Parser::error(const string& msg) const {
58     cout << "Syntax error: " << msg << endl;
59     return ParseNode("error", true);
60 }
61
62 // Ubah token menjadi label terminal parse tree.
63 string Parser::tokenToLabel(const Token& t) const {
64     string typeStr = tokenToString(t.type);
65     if (!t.value.empty()) {
66         return typeStr + "(" + t.value + ")";
67     }
68     return typeStr;
69 }
70
71 // Fungsi yang akan dipanggil di main
72 // Parse keseluruhan program sebagai root parse tree.
73 ParseNode Parser::parse() {
74     ParseNode result = ParseNode("<Program>");
75     result.children.push_back(parseProgramHeader());
76     result.children.push_back(parseDeclarationPart());
77     result.children.push_back(parseCompoundStatement());
78     result.children.push_back(expect(TokenType::PERIOD));
79
80     return result;
81 }
82
83
84

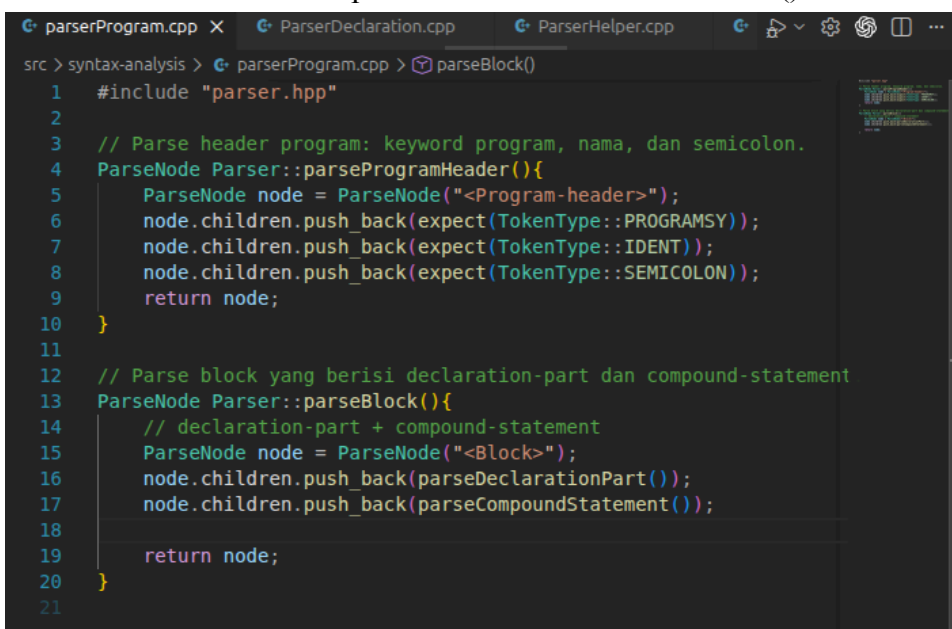
```

Gambar 2. Implementasi Parser() sebagai fungsi utama



```
ParserDeclaration.cpp X ParserHelper.cpp ParserType.cpp pars
src > syntax-analysis > ParserDeclaration.cpp > parseParameterGroup()
1  #include "parser.hpp"
2
3  // Parse seluruh bagian deklarasi sebelum compound statement.
4  ParseNode Parser::parseDeclarationPart(){
5      ParseNode node = ParseNode("<declaration-part>");
6
7      while (match(TokenType::CONSTSY)){
8          node.children.push_back(parseConstDeclaration());
9      }
10     while (match(TokenType::TYPESY)){
11         node.children.push_back(parseTypeDeclaration());
12     }
13     while (match(TokenType::VARSY)){
14         node.children.push_back(parseVarDeclaration());
15     }
16     while (match(TokenType::PROCEDURESY) || match(TokenType::FUNCT
17         node.children.push_back(parseSubprogramDeclaration());
18     }
19
20     return node;
21 }
22
```

Gambar 3. Implementasi ParserDeclarationPart()



```
parserProgram.cpp X ParserDeclaration.cpp ParserHelper.cpp
src > syntax-analysis > parserProgram.cpp > parseBlock()
1  #include "parser.hpp"
2
3  // Parse header program: keyword program, nama, dan semicolon.
4  ParseNode Parser::parseProgramHeader(){
5      ParseNode node = ParseNode("<Program-header>");
6      node.children.push_back(expect(TokenType::PROGRAMSY));
7      node.children.push_back(expect(TokenType::IDENT));
8      node.children.push_back(expect(TokenType::SEMICOLON));
9      return node;
10 }
11
12 // Parse block yang berisi declaration-part dan compound-statement
13 ParseNode Parser::parseBlock(){
14     // declaration-part + compound-statement
15     ParseNode node = ParseNode("<Block>");
16     node.children.push_back(parseDeclarationPart());
17     node.children.push_back(parseCompoundStatement());
18
19     return node;
20 }
21
```

Gambar 4. Implementasi ParserProgramHeader()

```

ParserStatement.cpp X parserProgram.cpp ParserDeclaration.cpp
src > syntax-analysis > ParserStatement.cpp > parseWhileStatement()
3 namespace {
23 }
24
25 // Parse block statement yang diawali begin dan diakhiri end.
26 ParseNode Parser::parseCompoundStatement(){
27     // beginsy + statement-list + endsy
28     ParseNode node = ParseNode("<compound-statement>");
29
30     node.children.push_back(expect(TokenType::BEGINSY));
31     node.children.push_back(parseStatementList());
32     node.children.push_back(expect(TokenType::ENDSY));
33
34     return node;
35 }
36
37 // Parse daftar statement yang dipisahkan semicolon.
38 ParseNode Parser::parseStatementList(){
39     // statement (semicolon + statement)*
40     ParseNode node = ParseNode("<statement-list>");
41
42     node.children.push_back(parseStatement());
43
44     while (match(TokenType::SEMICOLON)){
45         node.children.push_back(expect(TokenType::SEMICOLON));
46         node.children.push_back(parseStatement());
47     }
48
49     return node;
50 }
51
52 // Parse satu statement atau statement kosong.
53 ParseNode Parser::parseStatement(){
54     /* (assignment-statement | if-statement | case-statement | while-statement)
55     ParseNode node = ParseNode("<statement>");
56
57     if (match(TokenType::IDENT)){
58         bool assignmentStatement = false;
59         int lookahead = 1;

```

Gambar 5. Implementasi ParserCompoundStatement()

```

1 #ifndef PRINTTREE_HPP
2 #define PRINTTREE_HPP
3
4 #include "../syntax-analysis/parseTreeNode.hpp"
5 #include <iostream>
6 #include <fstream>
7 #include <string>
8
9 // Fungsi untuk mencetak tree
10 inline void printTreeHelper(std::ostream& os, const ParseNode& node, const std::string& prefix, bool isLast, bool isRoot) {
11     if (isRoot) {
12         os << node.label << "\n";
13     } else {
14         os << prefix << (isLast ? "└─ " : "├─ ") << node.label << "\n";
15     }
16
17     std::string nextPrefix = prefix;
18     if (!isRoot) {
19         nextPrefix += (isLast ? "    " : "│  ");
20     }
21
22     for (size_t i = 0; i < node.children.size(); ++i) {
23         printTreeHelper(os, node.children[i], nextPrefix, i == node.children.size() - 1, false);
24     }
25 }
26
27 // Fungsi utama yang memanggil fungsi pencetakan tree
28 inline void printTree(std::ostream& os, const ParseNode& root) {
29     printTreeHelper(os, root, "", true, true);
30 }
31
32 #endif
33

```

Gambar 6. Implementasi printTree()

```
src > main.cpp > main(int, char * [])
12
13 using namespace std;
14
15 int main(int argc, char* argv[]) {
16     if (argc == 1) {
17         return runInteractiveMode();
18     }
19
20     if (argc > 4) {
21         cout << "Usage: " << argv[0] << " [source_file] [lexical_output_file] [syntax_output_file]" << endl;
22         return 1;
23     }
24
25     return runCommandMode(argc, argv);
26 }
```

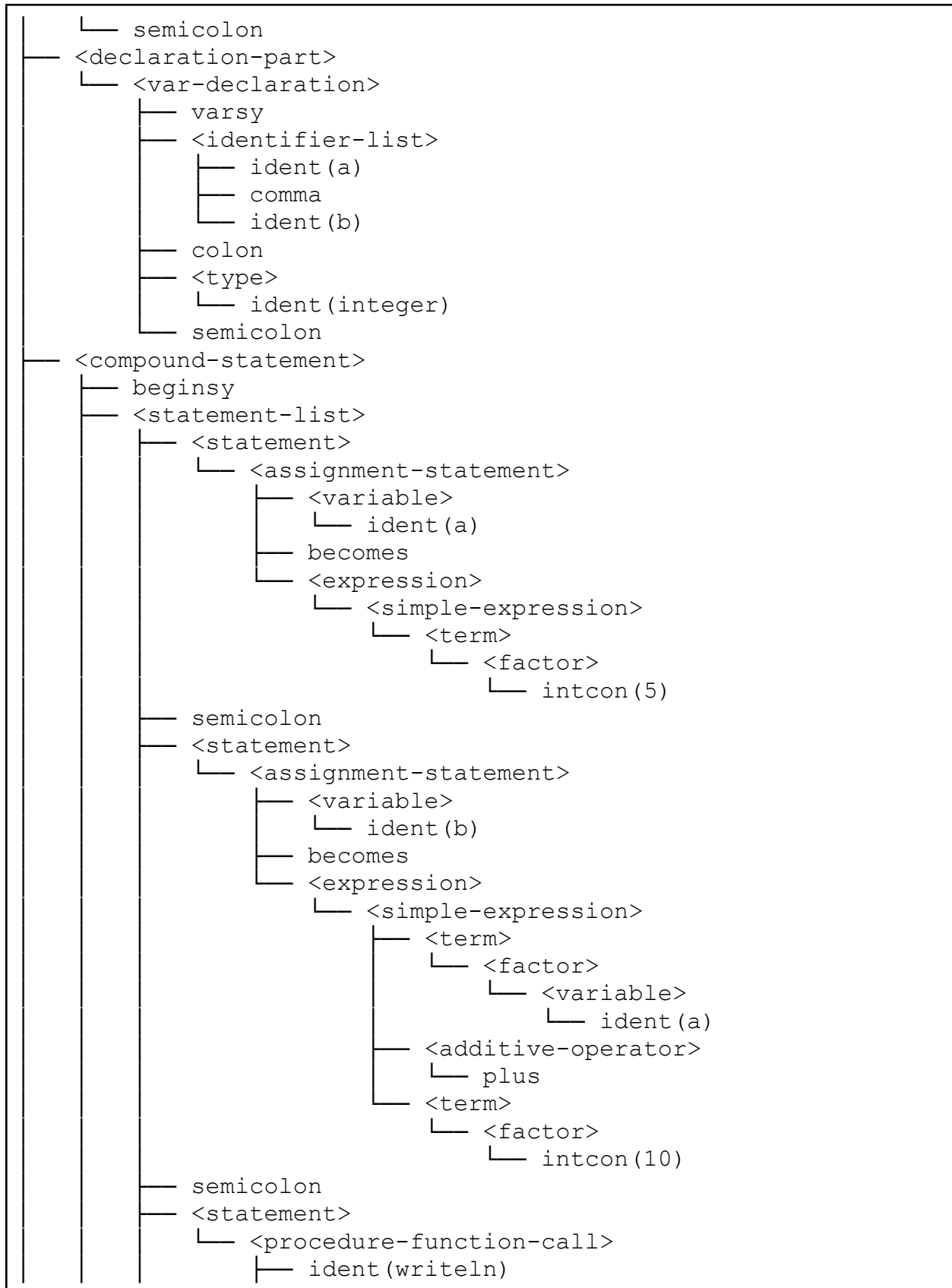
Gambar 7. Main

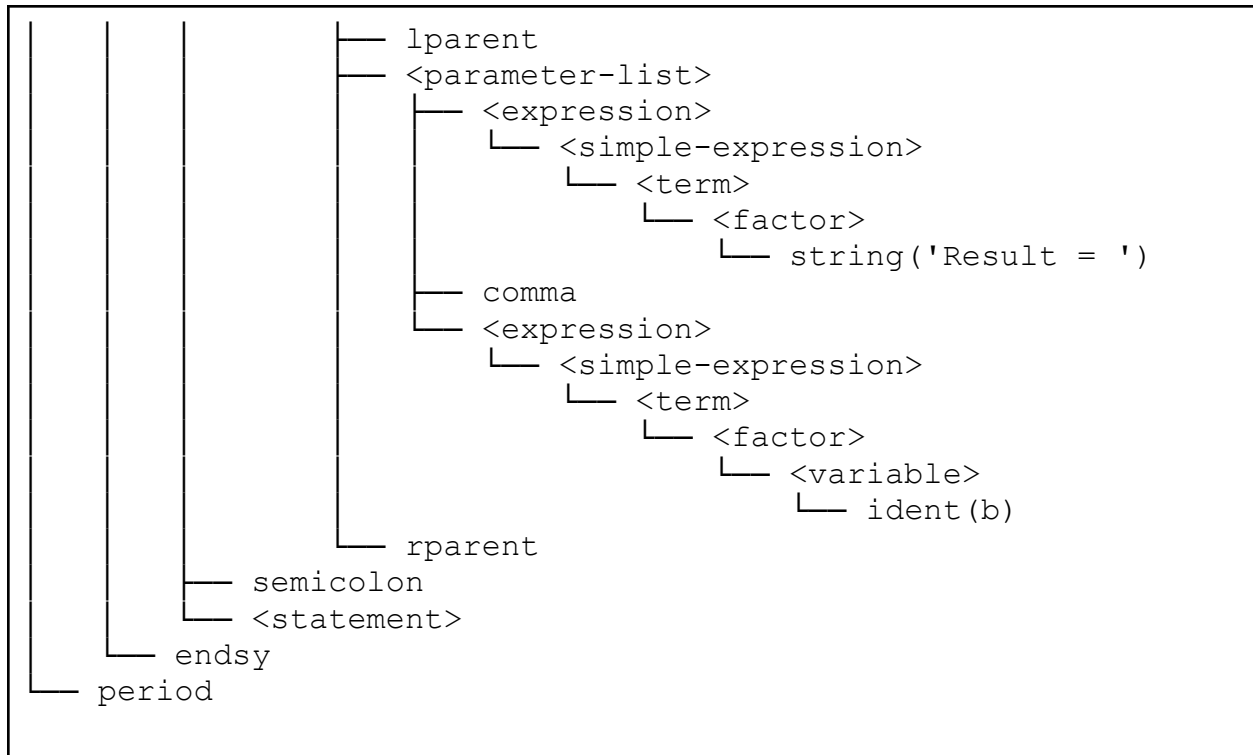
BAB 3 PENGUJIAN

*Catatan : Beberapa output memang memiliki font size yang berbeda karena ukuran docs yang terbatas.

3.1 Pengujian 1 (input-1.txt):

Masukan (input)
<pre>programsy ident (Hello) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus intcon (10) semicolon ident (writeln) lparent string ('Result = ') comma ident (b) rparent semicolon endsy period</pre>
Keluaran (output)
<pre><Program> ├─ <Program-header> │ └─ programsy │ └─ ident (Hello)</pre>





3.2 Pengujian 2 (input-2.txt):

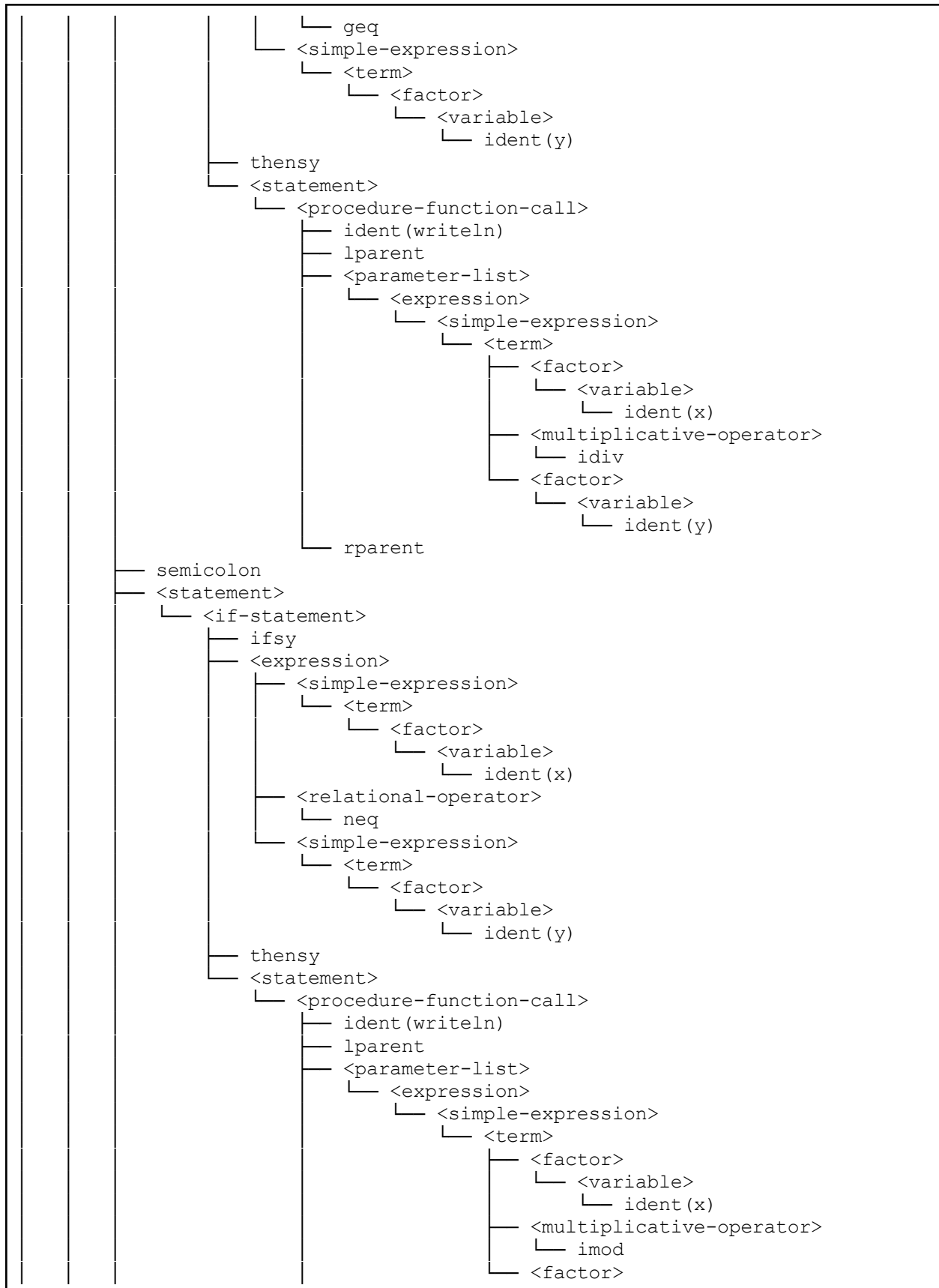
Masukan (input)
<pre> programsy ident(Operators) semicolon varsy ident(x) comma ident(y) colon ident(integer) semicolon beginsy ident(x) becomes intcon(10) semicolon ident(y) </pre>

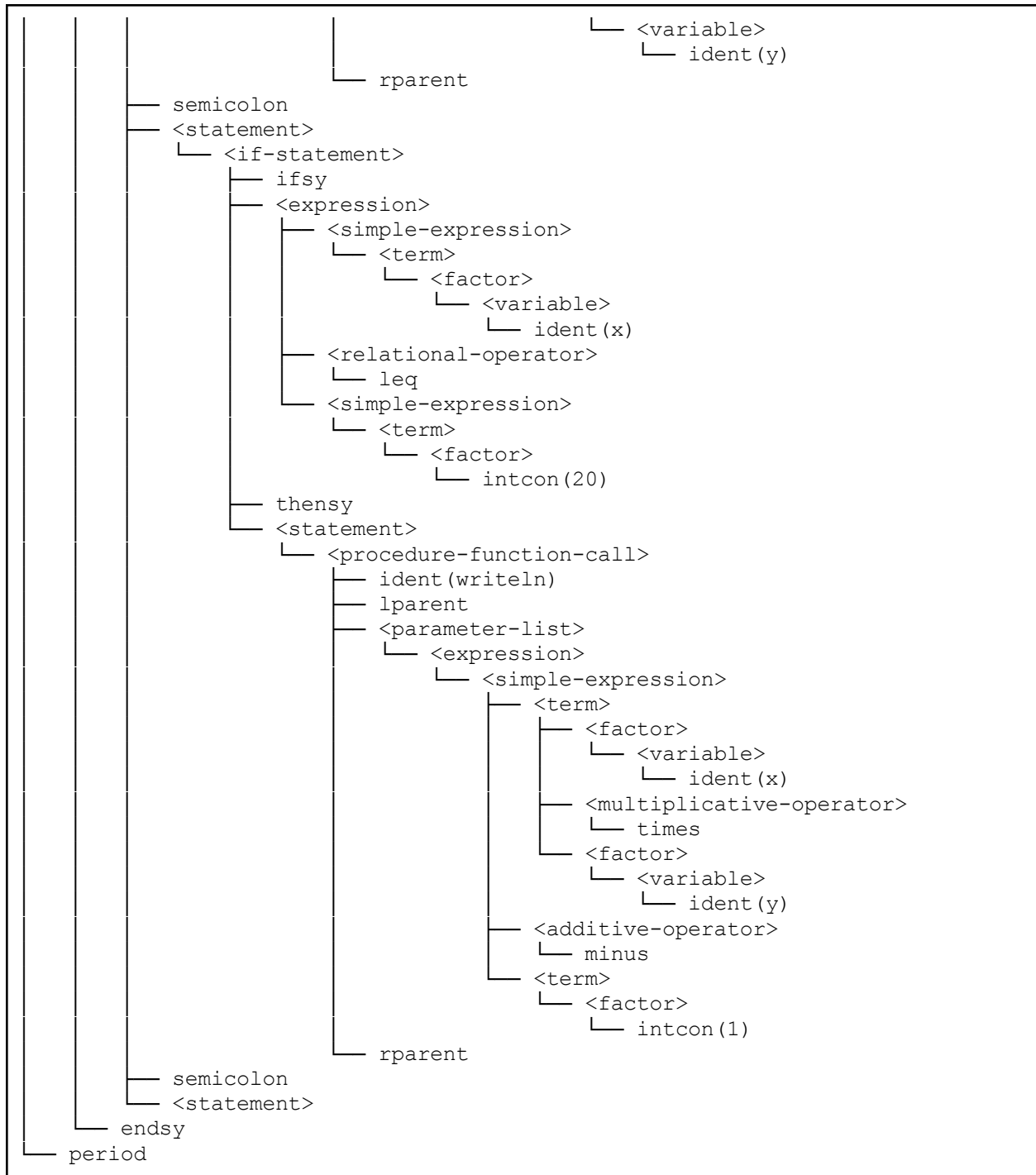
```
becomes
intcon(3)
semicolon
ifsy
ident(x)
geq
ident(y)
thensy
ident(writeln)
lparent
ident(x)
idiv
ident(y)
rparent
semicolon
ifsy
ident(x)
neq
ident(y)
thensy
ident(writeln)
lparent
ident(x)
imod
ident(y)
rparent
semicolon
ifsy
ident(x)
leq
intcon(20)
thensy
ident(writeln)
lparent
ident(x)
times
ident(y)
minus
intcon(1)
rparent
semicolon
endsy
period
```

Visualisasi pemrosesan dalam program

Keluaran (output)

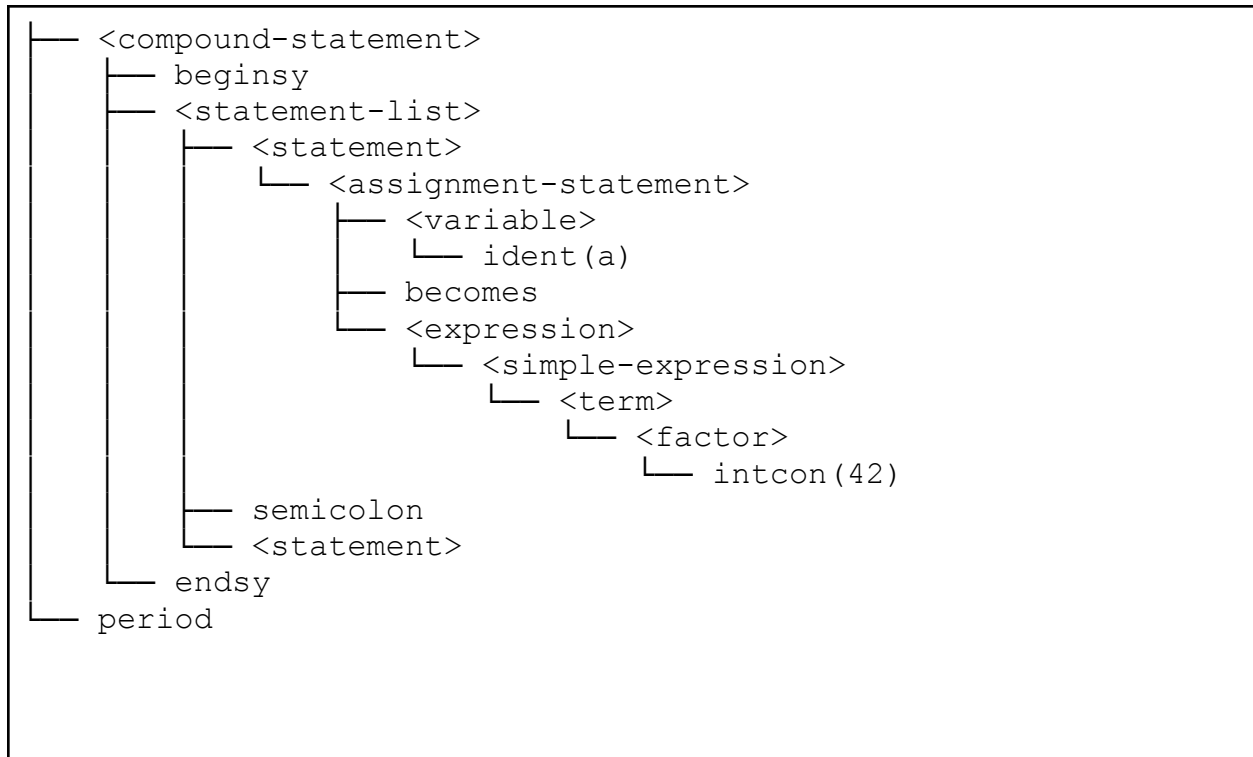
```
<Program>
├── <Program-header>
│   ├── programsy
│   ├── ident(Operators)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident(x)
│       │   ├── comma
│       │   └── ident(y)
│       ├── colon
│       ├── <type>
│       │   └── ident(integer)
│       └── semicolon
└── <compound-statement>
    ├── beginsy
    ├── <statement-list>
    │   ├── <statement>
    │   │   └── <assignment-statement>
    │   │       ├── <variable>
    │   │       │   └── ident(x)
    │   │       ├── becomes
    │   │       └── <expression>
    │   │           └── <simple-expression>
    │   │               └── <term>
    │   │                   └── <factor>
    │   │                       └── intcon(10)
    │   ├── semicolon
    │   ├── <statement>
    │   │   └── <assignment-statement>
    │   │       ├── <variable>
    │   │       │   └── ident(y)
    │   │       ├── becomes
    │   │       └── <expression>
    │   │           └── <simple-expression>
    │   │               └── <term>
    │   │                   └── <factor>
    │   │                       └── intcon(3)
    │   ├── semicolon
    │   └── <statement>
    │       └── <if-statement>
    │           ├── ifsy
    │           ├── <expression>
    │           │   └── <simple-expression>
    │           │       └── <term>
    │           │           └── <factor>
    │           │               └── <variable>
    │           │                   └── ident(x)
    │           └── <relational-operator>
```





3.3 Pengujian 3 (input-3.txt):

Masukan (input)
<pre>programsy ident(Comments) semicolon comment({ ini komentar kurung kurawal }) varsy ident(a) colon ident(integer) semicolon beginsy comment((* ini komentar bintang kurung *)) ident(a) becomes intcon(42) semicolon comment({ komentar lagi }) endsy period</pre>
Keluaran (output)
<pre><Program> ├── <Program-header> │ ├── programsy │ ├── ident(Comments) │ └── semicolon ├── <declaration-part> │ └── <var-declaration> │ ├── varsy │ ├── <identifier-list> │ │ └── ident(a) │ ├── colon │ ├── <type> │ │ └── ident(integer) │ └── semicolon</pre>



3.4 Pengujian 4 (input-4.txt):

Masukan (input)
programsy ident(Literals) semicolon varsy ident(c) colon ident(char) semicolon ident(s) colon ident(string) semicolon ident(r)

```

colon
ident(real)
semicolon
beginsy
ident(c)
becomes
charcon('A')
semicolon
ident(s)
becomes
string('Hello World')
semicolon
ident(r)
becomes
realcon(3.14)
semicolon
ident(writeln)
lparent
ident(c)
rparent
semicolon
ident(writeln)
lparent
ident(s)
rparent
semicolon
ident(writeln)
lparent
ident(r)
plus
realcon(100.05)
rparent
semicolon
endsy
period

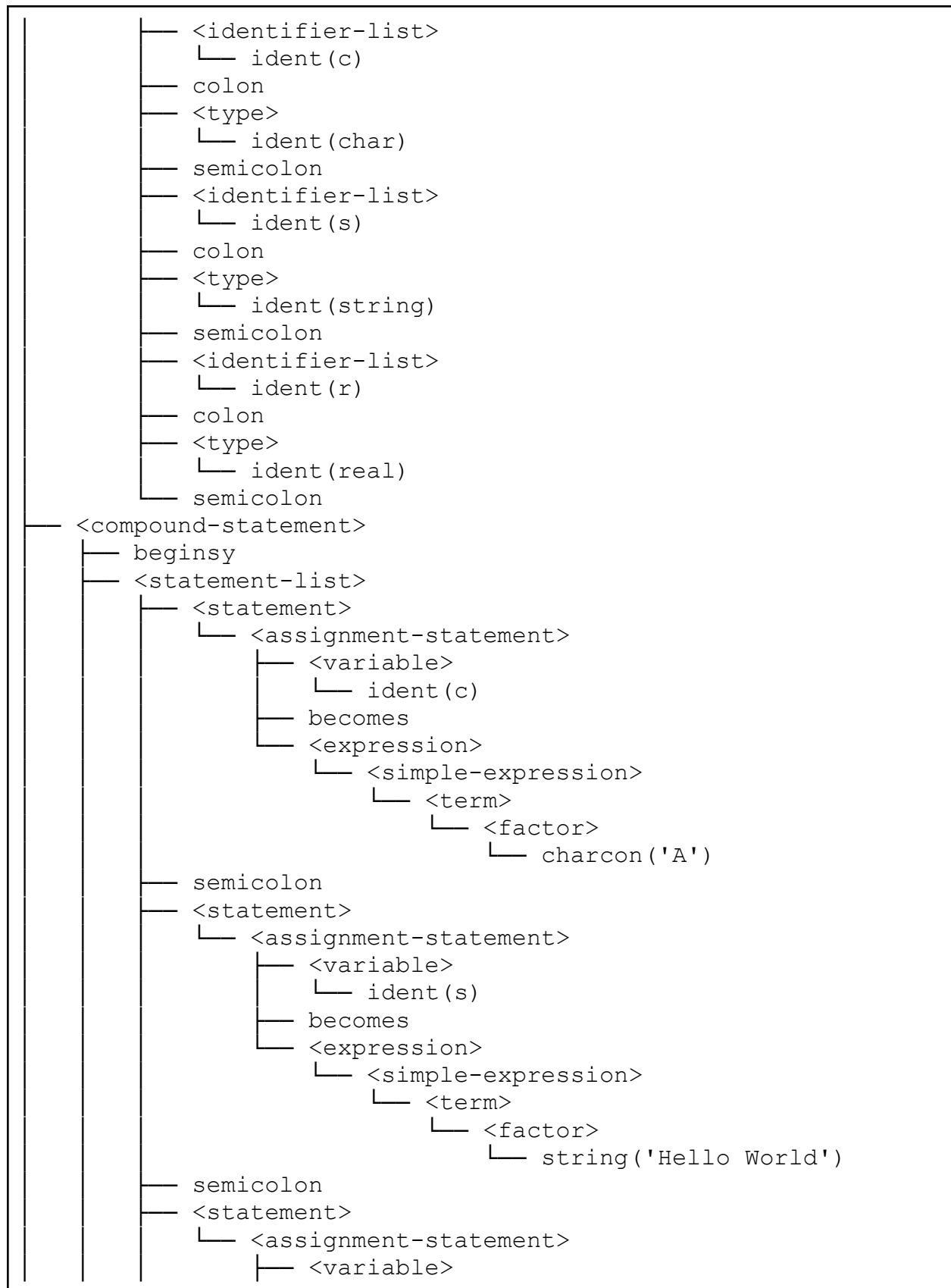
```

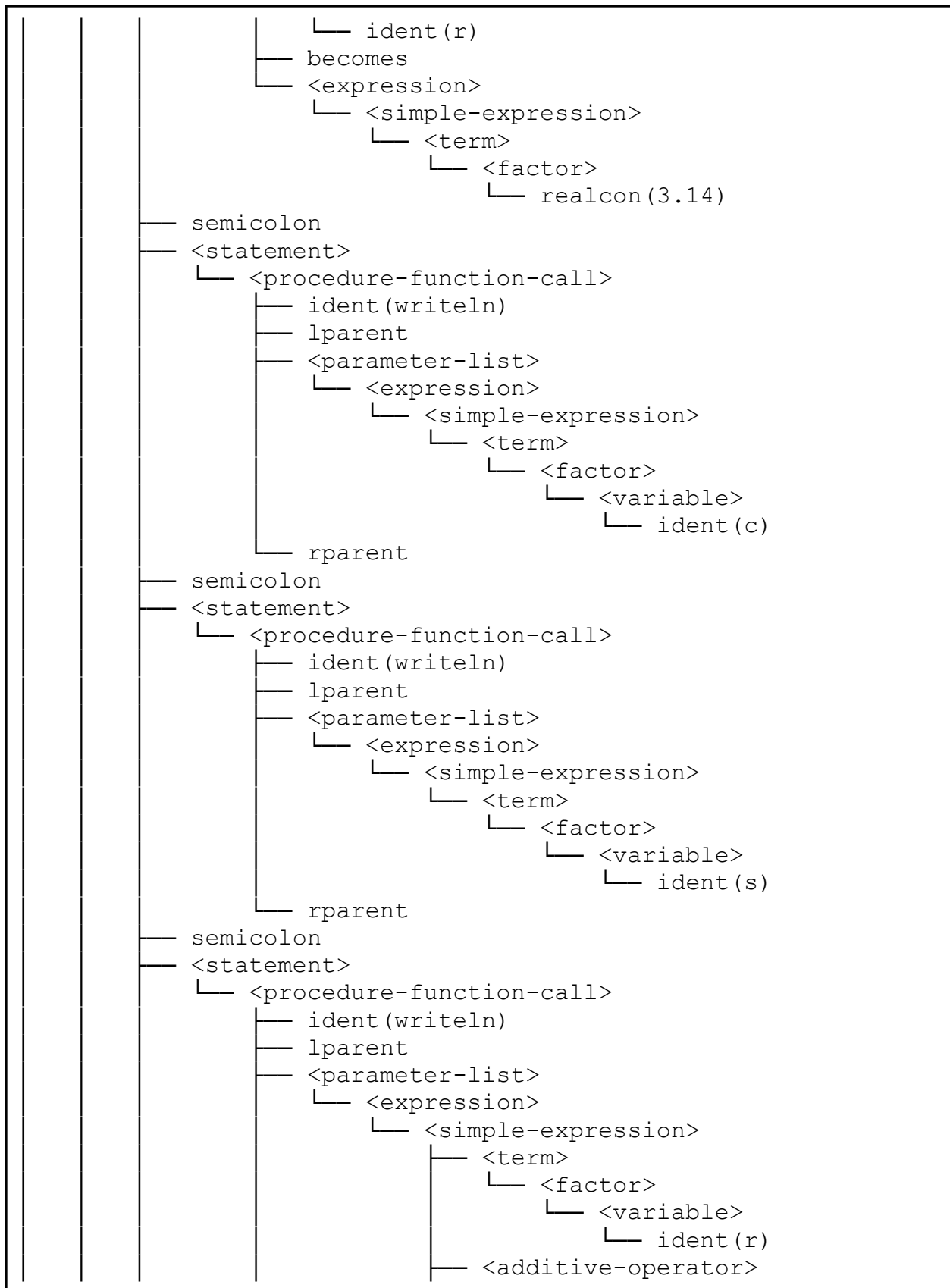
Keluaran (output)

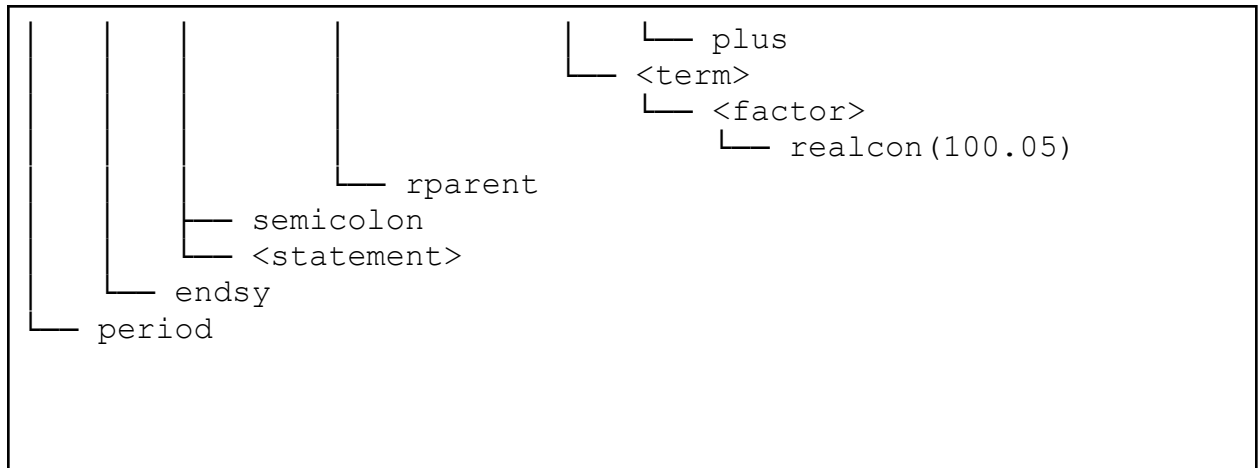
```

<Program>
├── <Program-header>
│   ├── programsy
│   ├── ident(Literals)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       └── varsy

```







3.5 Pengujian 5 (input-5.txt):

Masukan (input)
<pre>programsy ident (LoopArray) semicolon varsy ident (arr) colon arraysy lbrack intcon (1) period period intcon (5) rbrack ofsy ident (integer) semicolon ident (i) colon ident (integer) semicolon beginsy forsy ident (i) becomes intcon (1) tosy intcon (5) dosy ident (arr) lbrack ident (i) rbrack becomes ident (i) times intcon (2) semicolon ident (i) becomes intcon (1)</pre>

```

semicolon
repeatsy
ident(writeln)
lparent
ident(arr)
lbrack
ident(i)
rbrack
rparent
semicolon
ident(i)
becomes
ident(i)
plus
intcon(1)
semicolon
untilsy
ident(i)
eq1
intcon(5)
semicolon
whilesy
ident(i)
gtr
intcon(0)
dosy
ident(i)
becomes
ident(i)
minus
intcon(1)
semicolon
endsy
period

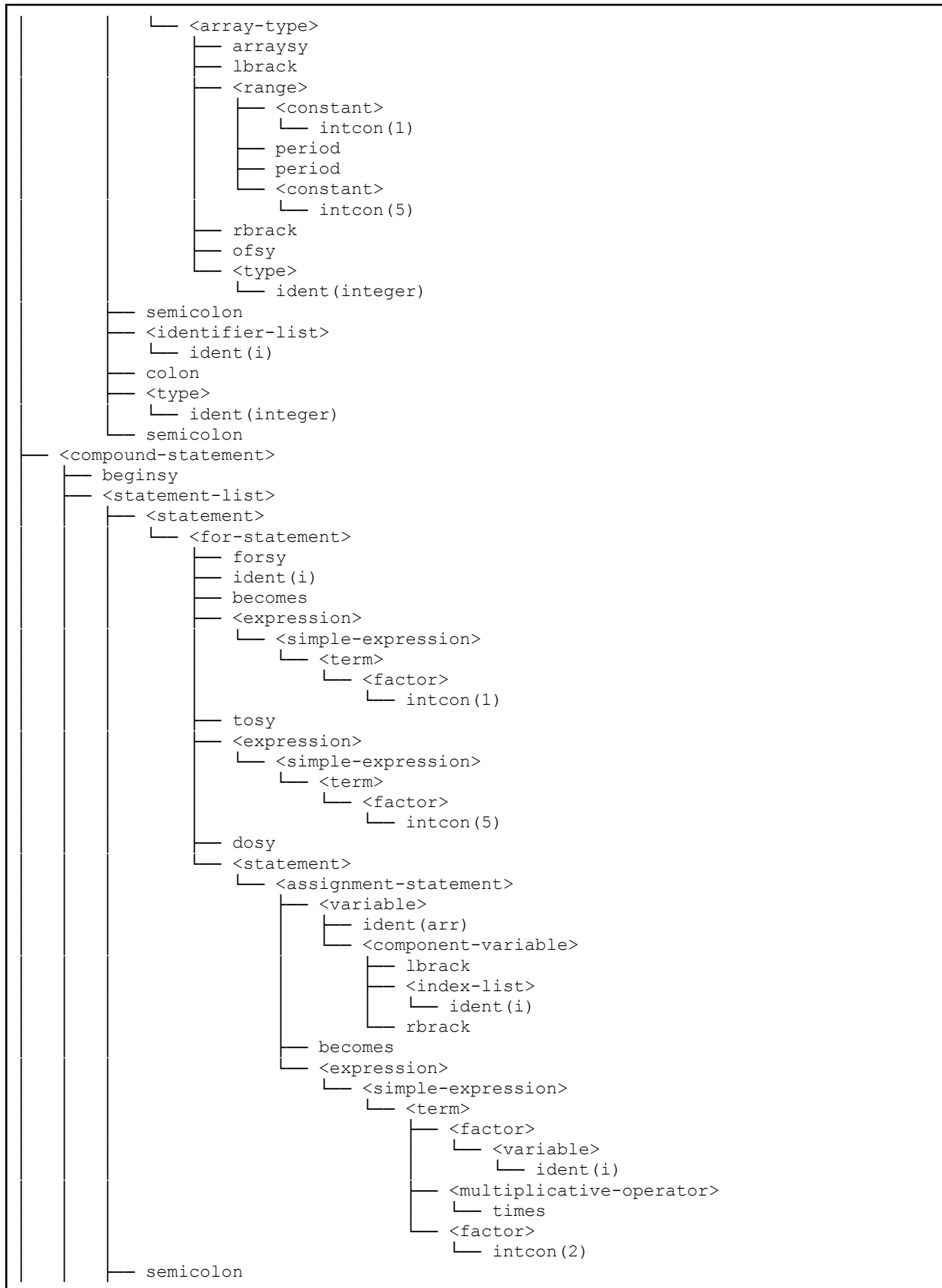
```

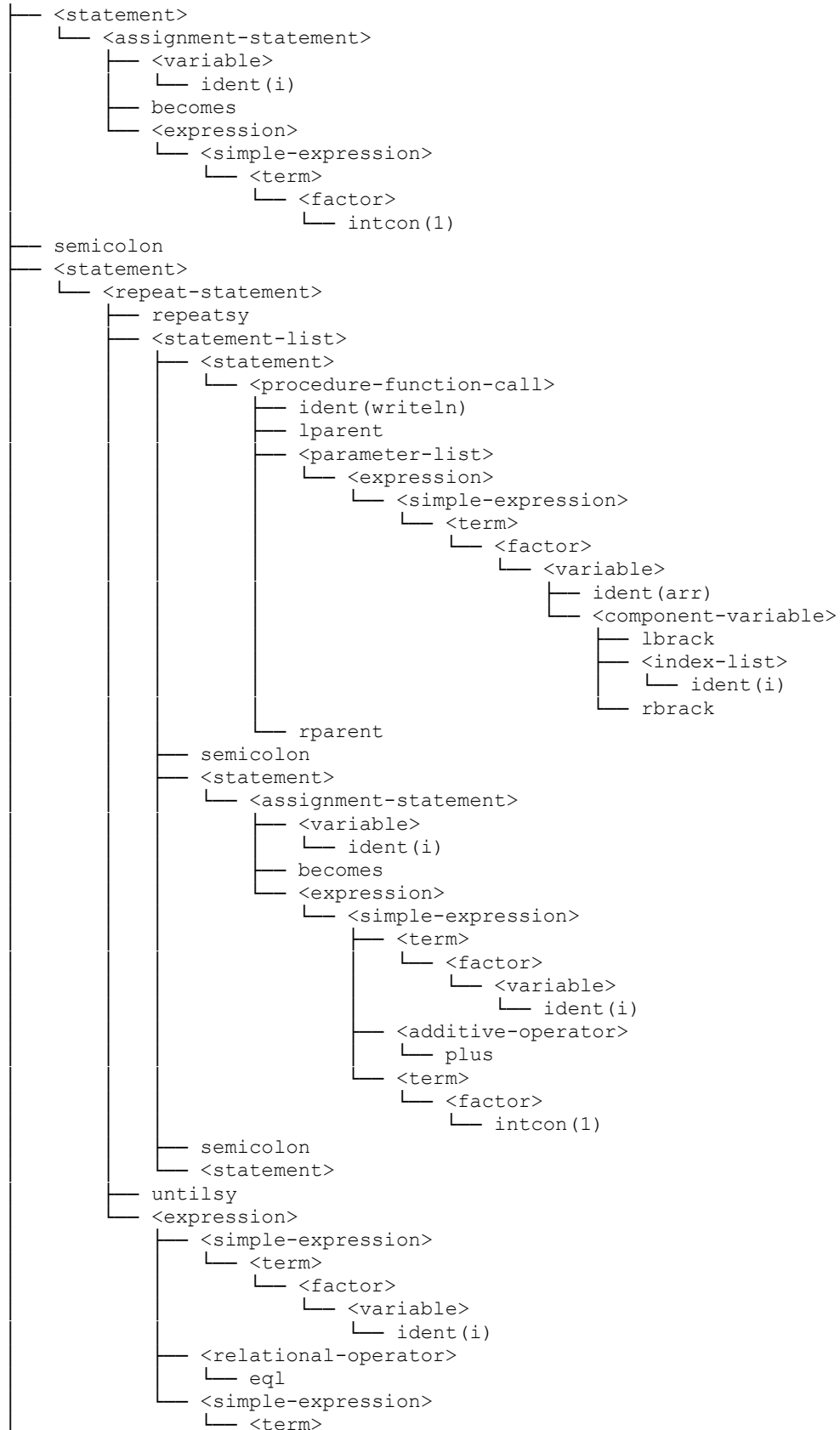
Keluaran (output)

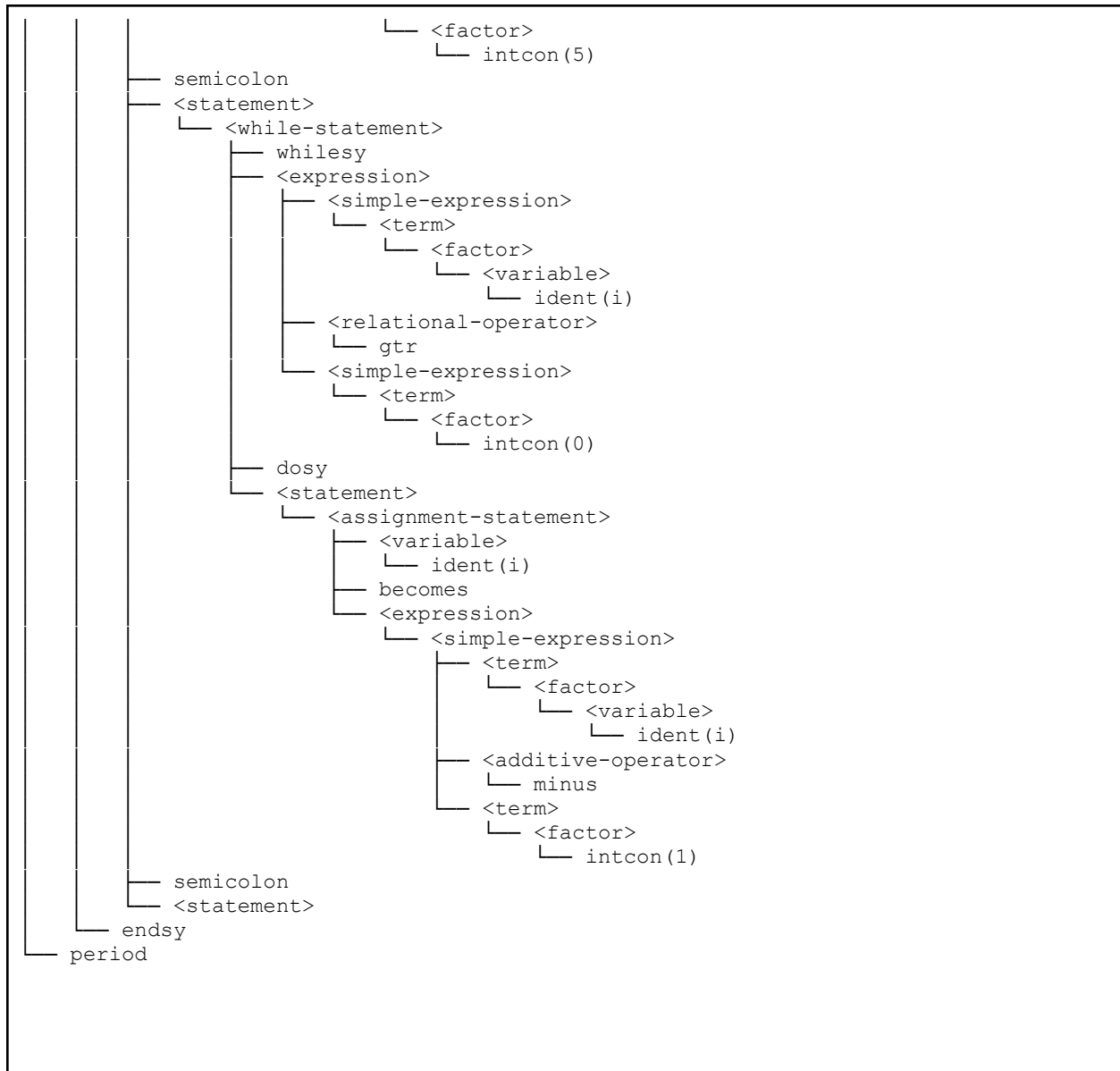
```

<Program>
├── <Program-header>
│   ├── programsy
│   ├── ident(LoopArray)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident(arr)
│       ├── colon
│       └── <type>

```







BAB 4 KESIMPULAN DAN SARAN

4.1 Kesimpulan

Kelompok std_abs telah berhasil merancang dan mengimplementasikan syntax analyzer untuk bahasa Arion. Parser dibangun secara modular. Parser mampu membaca list token hasil milestone 1, membangun parse tree lengkap dengan simbol terminal dan non-terminal sesuai grammar, serta mencetak pohon ke terminal dan ke file output. Seluruh kasus uji berhasil menghasilkan parse tree yang konsisten terhadap grammar bahasa Arion.

4.2 Saran.

- Pesan error disertai baris dan kolom. Token sudah memuat atribut line pada milestone 1, namun parser belum memanfaatkannya pada pesan error. Menyertakan line dan kolom akan sangat mempermudah debug input yang panjang.
- Pengembangan ke AST. Parse tree saat ini masih menyimpan seluruh simbol terminal. Untuk milestone berikutnya, parser dapat diperluas menghasilkan Abstract Syntax Tree yang lebih ringkas sebagai masukan tahap semantic analysis.
- Visualisasi otomatis. Modul astVisualizer sudah dapat menghasilkan file .dot; otomatisasi rendering ke PNG/SVG dalam pipeline build akan memudahkan dokumentasi dan debugging parse tree yang dalam.
- Pengujian tambahan. Menambahkan kasus uji negatif (input token yang sengaja salah) akan memverifikasi kualitas pesan error dan menjamin tidak ada skenario yang menyebabkan crash.

LAMPIRAN

Link Repository *Github* : <https://github.com/road2qnt/ABS-Tubes-IF2224-2026>

Pembagian Tugas

Nama	Pembagian Tugas	Kontribusi
Ega Luthfi Rais	Project Manager, Inisiasi GitHub, Design struktur gitHub, Assist Design dan Implementasi awal Recursive Descent	25%
Salman Faiz Assidqi	Laporan	25%
Ahmad Fauzan Putra	Laporan dan Testing	25%
Leonardus Brain Fatolosja	Laporan, Perancangan Program, Implementasi Parser dengan menggunakan Recursive Descent dan Testing	25%

REFERENSI

GeeksforGeeks. (2025). Recursive Descent Parser. Diakses dari

<https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>

GeeksforGeeks. (2025). Parse Tree in Compiler Design. Diakses dari

<https://www.geeksforgeeks.org/compiler-design/parse-tree-in-compiler-design/>

GeeksforGeeks. (2026). Introduction to Syntax Analysis in Compiler Design. Diakses dari

<https://www.geeksforgeeks.org/compiler-design/introduction-to-syntax-analysis-in-compiler-design/>

GeeksforGeeks. (2023). Abstract Syntax Tree vs Parse Tree. Diakses dari

<https://www.geeksforgeeks.org/compiler-design/abstract-syntax-tree-vs-parse-tree/>

GeeksforGeeks. (2025). Types of Parsers in Compiler Design. Diakses dari

<https://www.geeksforgeeks.org/compiler-design/types-of-parsers-in-compiler-design/>